BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. No. Price is not the same as licence/source rights.

Q2. Sum 0 and carry-out 1.

Q3. 10.

Q4. A link/reference to the next node (for a singly linked list).

You Do pages · Independent questions

Q1. Compiler high-level source to target code [2], interpreter execution-time evaluation [2], assembler assembly mnemonics to machine/object code [2].

Q2. Potential runtime performance [2], distribution without original source [2], early compile-time error detection/optimisation [2].

Q3. Correct translator characteristics [3], specific debugging trade-off [2], portability/target dependence [2], qualified conclusion [1].

Exit Ticket · Model answer

False. Assemblers translate assembly language.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 114 · Lesson 114 · Compilers, Interpreters and Assemblers

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Does free-of-charge imply that software is open source?

Q2. A full adder receives A=1, B=1 and carry-in=0. What are sum and carry-out?

Q3. For the binary value 1010, what is its decimal value?

Q4. A linked-list node has data and what other essential field?

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

compiler — Translates source to target code before execution.

interpreter — Executes by translating/evaluating at runtime.

assembler — Turns assembly into machine/object code.

object code — Compiled code awaiting linking or execution.

New learning

Compiler — Translates a complete source unit to target code before execution, often enabling optimisation.

Interpreter — Executes by translating/evaluating during running; behaviour varies by implementation.

Assembler — Converts symbolic assembly mnemonics into machine instructions.

Worked example

A team distributes an executable without its original C source. Explain the compiler's role.

1. Model answer — A compiler translates source to target machine/object code; linked executable can run without the source being present. Compilation can report syntax errors and apply optimisation before execution. — Check: link each claim to the scenario and use precise OCR terminology.

We Do · Practise with guidance

A technician reads LMC 'LDA 20' and wants the CPU's numeric instruction. What translation tool is used?

What must we decide?

Identify the inputs, relevant facts and exact output or judgement required.

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

An assembler converts the mnemonic plus operand into its encoded machine instruction. In standard LMC, LDA 20 encodes as 520. This is not an interpreter translating a high-level language.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Distinguish compiler, interpreter and assembler with a relevant input for each. [6]

Question 2

Explain two advantages of compiling a program before distribution. [6]

Question 3

A Python-like runtime, C compiler and LMC assembler are used in one project. Compare debugging, portability and execution trade-offs. [8]

Exit Ticket · Answer before checking

An assembler translates high-level Python syntax into binary machine code.